University of Tehran

Faculty of ECE

# Digital systems design

## CA1&2

Reza mohamadi

810103509

Spring 1405

# 1.CMOS Switch-Level Design of a 3-Input Zero's Counter

## 1.1 Objective

The objective of this section is to design a 3-input Zero's Counter (ZC) at the CMOS switch level using complex gates. The design requires minimizing the transistor count while considering the inherent inversion delay of CMOS gates. The circuit takes three inputs (a, b, and c) and outputs a 2-bit binary number ($y_1y_0$) representing the total number of zeros present at the inputs.

## 1.2 Truth Table

Based on the definition of a 3-input Zero's Counter, the truth table is constructed as follows:

| a b c | **Number of Zeros** | $y_1$**(MSB)** | $y_0$**(LSB)** |
|---|---|---|---|
| 0 0 0 | 3 | 1 | 1 |
| 0 0 1 | 2 | 1 | 0 |
| 0 1 0 | 2 | 1 | 0 |
| 0 1 1 | 1 | 0 | 1 |
| 1 0 0 | 2 | 1 | 0 |
| 1 0 1 | 1 | 0 | 1 |
| 1 1 0 | 1 | 0 | 1 |
| 1 1 1 | 0 | 0 | 0 |

## 1.3 Boolean Algebra and Logic Derivation

From the truth table, we extract the unminimized Boolean expressions for the outputs $y_0$and $y_1$.

**For $y_0$(LSB):**

The output $y_0$is high when the number of zeros is odd (1 or 3).

$$y_0 = \bar{a}\bar{b}\bar{c} + \bar{a}bc + a\bar{b}c + ab\bar{c}$$

This expression represents the logical XNOR of the three inputs:

$$\mathrm{y}_0 = \overline{a \oplus b \oplus c}$$

To prepare this for a CMOS complex gate, we can factor the expression by grouping terms with cand $\bar{c}$:

$$y_0 = \mathrm{c}\left(ab + \bar{a}\bar{b}\right) + \bar{c}\left(a\bar{b} + \bar{a}b\right)$$

**For $y_1$(MSB):**

The output $y_1$is high when there are 2 or 3 zeros (which is the majority function for zeros).

$$y_1 = \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + \bar{a}b\bar{c} + a\bar{b}\bar{c}$$

Simplifying using Boolean algebra yields:

$$y_1 = \bar{a}\bar{b} + \bar{b}\bar{c} + \bar{a}\bar{c}$$

Factoring to reduce transistor count:

$$y_1 = \bar{a}\bar{b} + \bar{c}(\bar{a} + \bar{b})$$

## 1.4 CMOS Complex Gate Optimization

Standard CMOS complex gates inherently invert their logic. Implementing $y_1$and $y_0$directly in the Pull-Up Network (PUN) would yield $\bar{y_1}$and $\bar{y_0}$at the output, requiring an additional inverter.

To optimize the transistor count and eliminate the need for the final inverters, we design the Pull-Down Network (PDN) using the *complement* of our desired outputs ($\bar{y_1}$ and $\bar{y_0}$). Due to the natural inversion of the CMOS gate, the final output node will directly provide $y_1$and $y_0$.

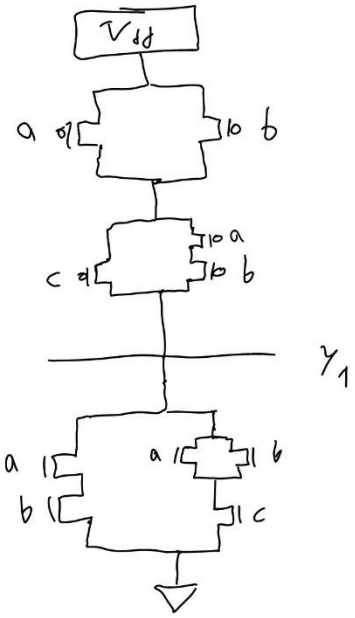
**1. Designing $y_1$(MSB):**

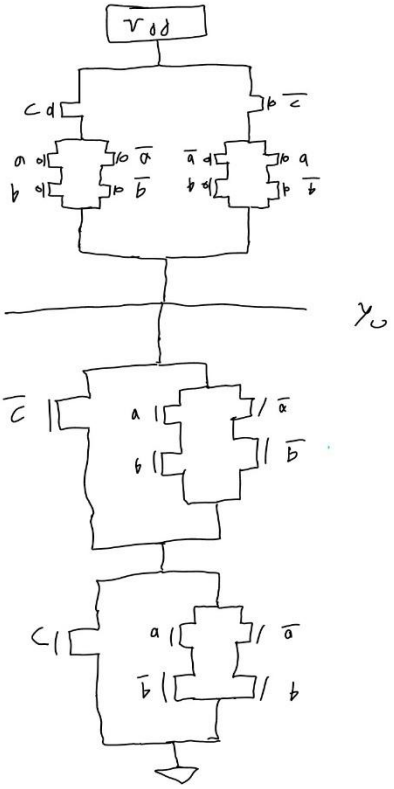
- **Complement logic ($\bar{y_1}$):** $\mathrm{ab} + \mathrm{ac} + \mathrm{bc} = \mathrm{ab} + \mathrm{c}(a + b)$
- **PDN (NMOS):** $\mathrm{ab} + \mathrm{c}(a + b)$
- **PUN (PMOS):** Applying De Morgan's laws (dual of PDN), we get: $(a + b) \cdot (c + ab)$

**2. Designing $y_0$(LSB):**

- **Complement logic ($\bar{y_0}$):**
$$y_0 = \overline{\overline{a \oplus b \oplus c}} = a \oplus b \oplus c$$
- Expanded complement logic: $\mathrm{c}(a\bar{b} + \bar{a}b) + \bar{c}(ab + \bar{a}\bar{b})$
- **PDN (NMOS):** $\mathrm{c}(a\bar{b} + \bar{a}b) + \bar{c}(ab + \bar{a}\bar{b})$
- **PUN (PMOS):** Applying De Morgan's law $\left(c + (a\bar{b} + \bar{a}b)\right) \cdot \left(\bar{c} + (ab + \bar{a}\bar{b})\right)$

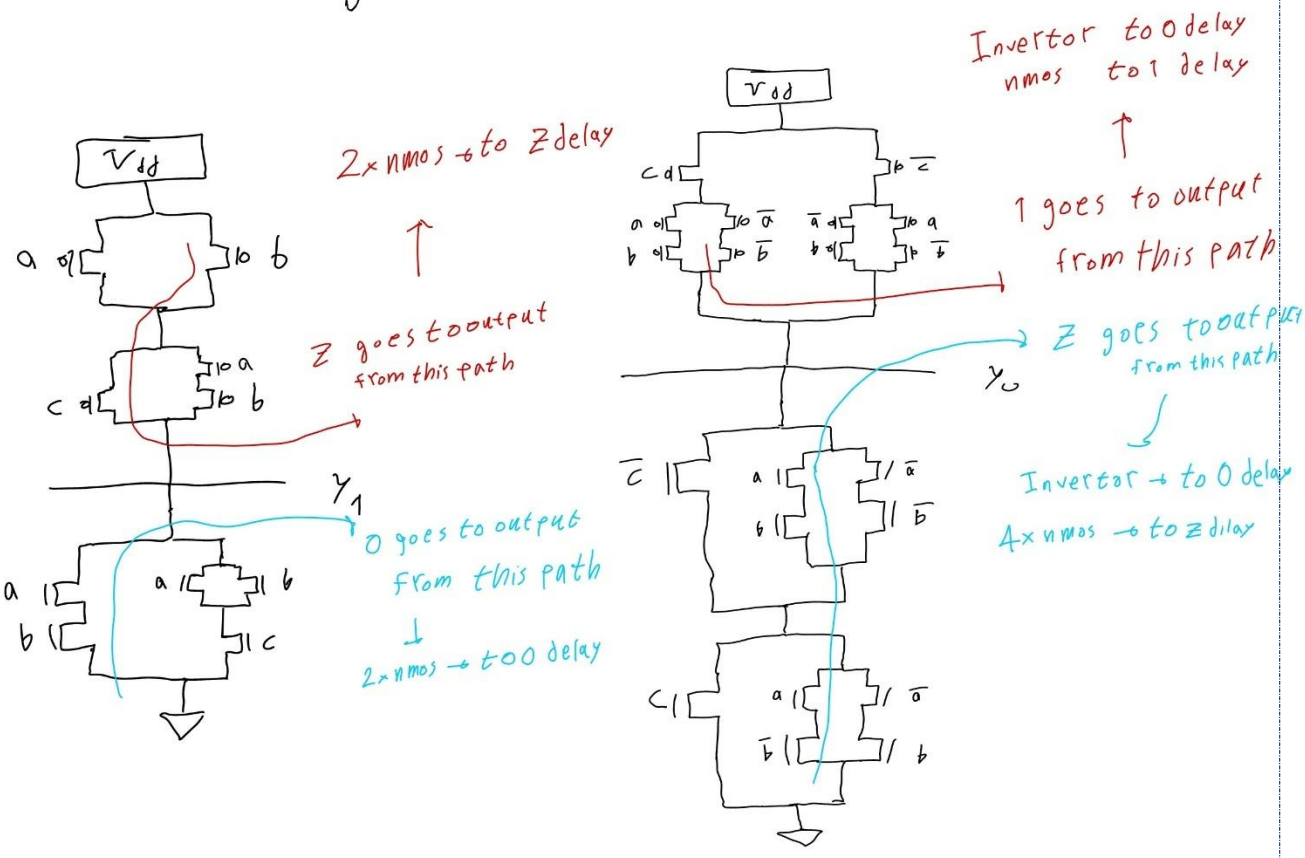
## 1.5 Transistor-Level Schematic and Delay Modeling and Adjustment





## 1.6 Delay Modeling and Adjustment

for worst case if we go from abc = 100 to abc = 110 :



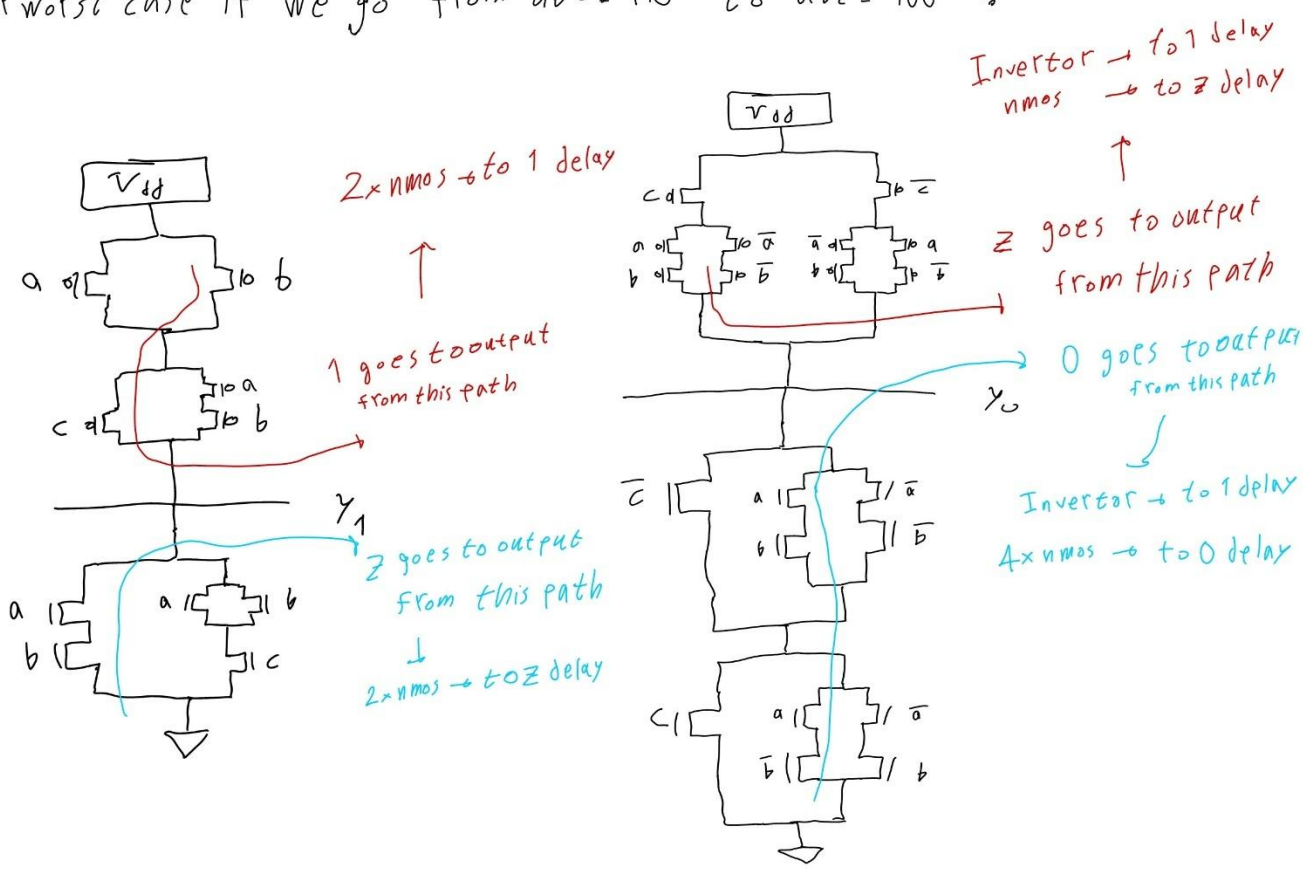

so ⟹ $Y_0$ delay : max( 7+5 , 4×5+7 ) = 27ns

$Y_1$ delay : max ( 2×7 , 2×4 ) = 14ns

⟹ structure dilay = max(27,14) = 27ns

⟹ so the worst case delay of our structur for this event is : 27ns

for worst case if we go from abc = 110 to abc = 100 :




so ⟹ $Y_0$ delay : max( 5+ 7 , 4×4+5 ) = 21 ns

$Y_1$ delay : max ( 2×5 , 2×5 ) = 10 ns

⟹ structure delay = max ( 21, 10 ) = 21 ns

⟹ so the worst case delay of our structur fo this event is : 21 ns

⟹ the worst case delay of our structur is :

$$\max(27, 21) = 27\,ns$$

## 2. Switch-Level Design of a 3-Input Zero's Counter

### 2.1 Objective

The objective of this section is to implement the 3-input Zero's Counter (ZC) circuit using Verilog switch-level primitives (`pmos` and `nmos`). The design requires modeling specific transistor delays: `#(3,4,5)` for nMOS and `#(5,6,7)` for pMOS. Finally, a testbench will be created to apply test vectors, and the resulting simulation will be analyzed to justify the output waveforms, paying special attention to any $X$(unknown) or $Z$(high impedance) states that occur

### 2.2 Verilog Implementation and Delay Modeling

The schematics were translated into Verilog using the built-in `pmos` and `nmos` primitives.

```
`timescale  1ns/1ns

module  Zc_complexGate (input a,b,c , output y1,y0);
    supply1 vdd;
    supply0 gnd;
    wire nodes[0:14];

    wire c_inv,a_inv,b_inv;
    pmos #(5,6,7) T1(c_inv,vdd,c);
    nmos #(3,4,5) T2(c_inv,gnd,c);

    pmos #(5,6,7) T3(a_inv,vdd,a);
    nmos #(3,4,5) T4(a_inv,gnd,a);

    pmos #(5,6,7) T5(b_inv,vdd,b);
    nmos #(3,4,5) T6(b_inv,gnd,b);

    pmos #(5,6,7) T7(nodes[0],vdd,a);
    pmos #(5,6,7) T8(nodes[0],vdd,b);
    pmos #(5,6,7) T9(nodes[1],nodes[0],a);
    pmos #(5,6,7) T10(y1,nodes[1],b);
    pmos #(5,6,7) T11(y1,nodes[0],c);

    nmos #(3,4,5) T13(nodes[2],gnd,b);
    nmos #(3,4,5) T14(y1,nodes[2],a);
    nmos #(3,4,5) T15(nodes[3],gnd,c);
    nmos #(3,4,5) T16(y1,nodes[3],b);
    nmos #(3,4,5) T17(y1,nodes[3],a);

    pmos #(5,6,7) T18(nodes[4],vdd,c);
    pmos #(5,6,7) T19(nodes[5],vdd,c_inv);
    pmos #(5,6,7) T20(nodes[6],nodes[5],a);
    pmos #(5,6,7) T21(nodes[13],nodes[5],a_inv);
```

```
    pmos #(5,6,7) T22(y0,nodes[13],b);
    pmos #(5,6,7) T23(y0,nodes[6],b_inv);
    pmos #(5,6,7) T24(nodes[7],nodes[4],a);
    pmos #(5,6,7) T25(nodes[14],nodes[4],a_inv);
    pmos #(5,6,7) T26(y0,nodes[7],b);
    pmos #(5,6,7) T27(y0,nodes[14],b_inv);


    nmos #(3,4,5) T28(nodes[8],gnd,c);
    nmos #(3,4,5) T29(y0,nodes[8],c_inv);
    nmos #(3,4,5) T30(nodes[9],gnd,b);
    nmos #(3,4,5) T31(nodes[10],gnd,b_inv);
    nmos #(3,4,5) T32(nodes[8],nodes[10],a);
    nmos #(3,4,5) T33(nodes[8],nodes[9],a_inv);
    nmos #(3,4,5) T34(nodes[11],nodes[8],b);
    nmos #(3,4,5) T35(nodes[12],nodes[8],b_inv);
    nmos #(3,4,5) T36(y0,nodes[11],a);
    nmos #(3,4,5) T37(y0,nodes[12],a_inv);

endmodule
```

### 2.5 Simulation and Analysis of Results

**Test bench code:**

```
`timescale 1ns/1ns

module Zc_complexGateTb;

    reg a;
    reg b;
    reg c;
    wire y1,y0;
    Zc_complexGate DUT (a,b,c,y1,y0);

    initial begin
        a = 0; b = 0; c = 0;
        #60;

        a = 1; b = 0; c = 0;
        #60;

        a = 1; b = 1; c = 0;
        #60;

        a = 0; b = 1; c = 0;
        #60;
```

```
        a = 0; b = 1; c = 1;
        #60;

        a = 0; b = 0; c = 1;
        #60;

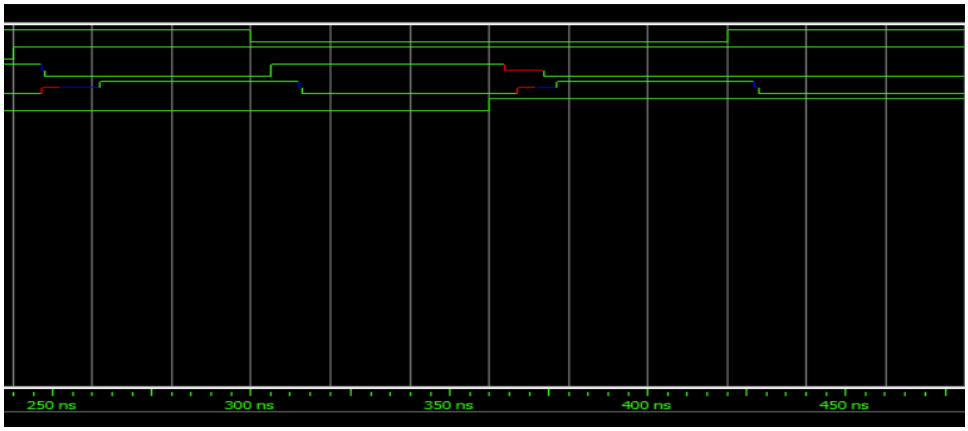
        a = 1; b = 0; c = 1;
        #60;

        a = 1; b = 1; c = 1;
        #60;
        $stop;
    end

endmodule
```

**simulation graph:**



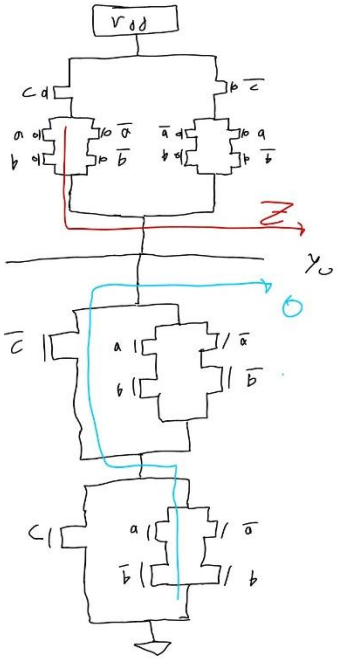
**Analysis of Transient States (X and Z) and justify the outputs.**

**We do this for first three transition.**

**First transition in 60ns : abc = 000 → 100**

abc : 000 → 100 ⇒ $Y_1 Y_0$ : 11 → 10

output justified based on simulation result




$Y_0$ is has to go from 1 to 0 in this transition.

pull down: after 4 + 4 = 8ns drive the output to 0.

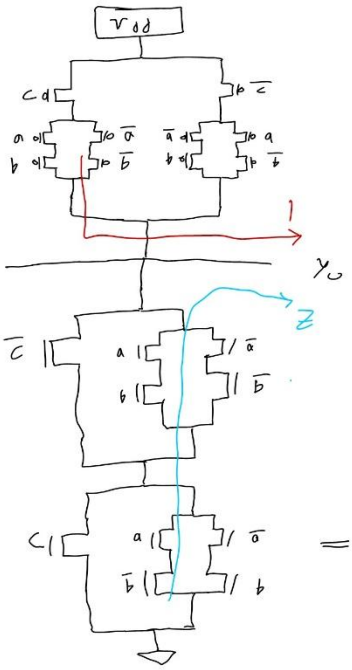
pull up: after 7 + 7 = 14ns drive the Z to output

⇒ so from t = 8ns to t = 14ns after the transition both pull up and pull down are conducting and we have X on output for $Y_0$

**Second transition in 120ns : abc = 100 → 110**

abc : 100 → 110 ⇒ $Y_1 Y_0$ : 10 → 01

output justified based on simulation result

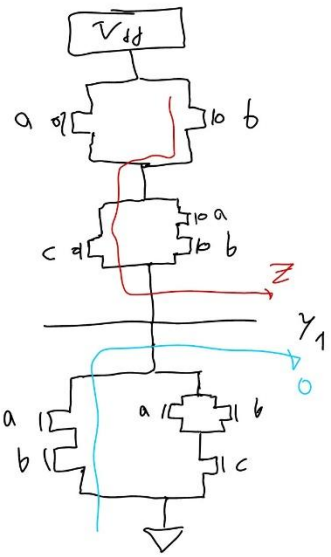



$Y_0$ is has to go from 0 to 1 in this transition:

pull down: after Inverter 7 + 4×5 = 27ns drive the output to Z

pull up: after 7 + 5 = 12ns drive the output to 1

⇒ so from t = 12ns to t = 27ns after the transition both pull up and pull down are conducting and we have X on output for $Y_0$




$Y_1$ is has to go from 1 to 0 in this transition:

pull down: after 4 + 4 = 8ns drive the output to 0

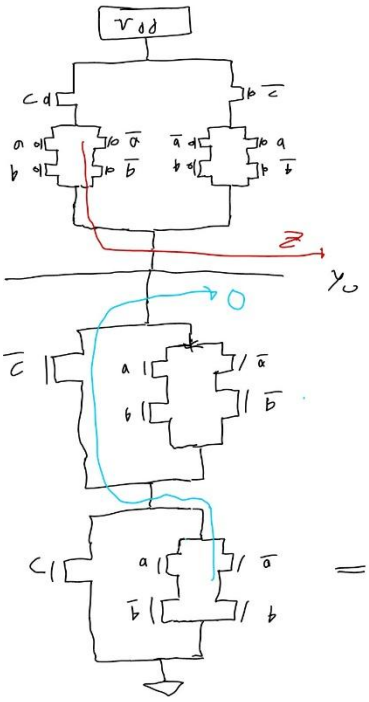
pull up: after 7 + 7 = 14ns drive the output to Z

⇒ so from t = 8ns to t = 14ns after the transition both pull up and pull down are conducting and we have X on output for $Y_1$

**third transition in 180ns : abc = 110 → 010**

abc : 110 → 010 ⇒ $Y_1 Y_0$ : 01 → 10

output justified based on simulation result

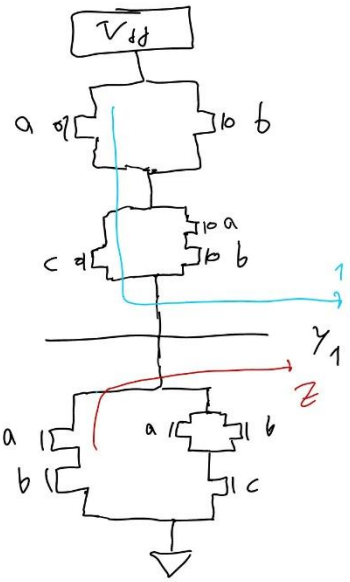



$Y_0$ is has to go from 1 to 0 in this transition:

pull down: after 5 + 2×4 = 13 ns drive the output to 0

pull up: after 5 + 2×7 = 19 ns drive the output to Z

⇒ so from t = 13 ns to t = 19 ns after the transition both pull up and pull down are conducting and we have X on output for $Y_0$




$Y_1$ is has to go from 0 to 1 in this transition:

pull down: after 5 ns drive the output to Z

pull up: after 2 × 5 = 10 ns drive the output to 1

⇒ so from t = 5 ns to t = 10 ns after the transition both pull up and pull down are disconnect so the output for $Y_1$ is Z during that time

# 3. Gate-Level Design of a 3-Input Zero's Counter Using Verilog Primitives

### 3.1 Objective

The objective of this section is to describe the 3-input Zero's Counter (ZC) designed in Part 2 using standard Verilog gate primitives .

### 3.2 Logic Implementation Strategy

To implement the circuit using standard gate primitives while maintaining functional equivalence to the previous parts, the unminimized Boolean expressions were adapted to utilize NOR and XOR/NOT structures.

**For $y_0$(LSB):**

The logic for $y_0$represents the XNOR function of the three inputs.

$$y_0 = \overline{(a \oplus b) \oplus c}$$

*Implementation details:* This is realized by cascading two 2-input XOR gates (where the XOR gates themselves are constructed from standard NAND primitives) and passing the final output through a NOT gate to achieve the XNOR function.

**For $y_1$(MSB):**

The expression $y_0 = \bar{a}\bar{b} + \bar{a}\bar{c} + \bar{b}\bar{c}$ can be rewritten using De Morgan's laws to be implemented with standard primitives:

$$y_1 = \overline{\overline{(a+b)} + \overline{(a+c)} + \overline{(b+c)}}$$

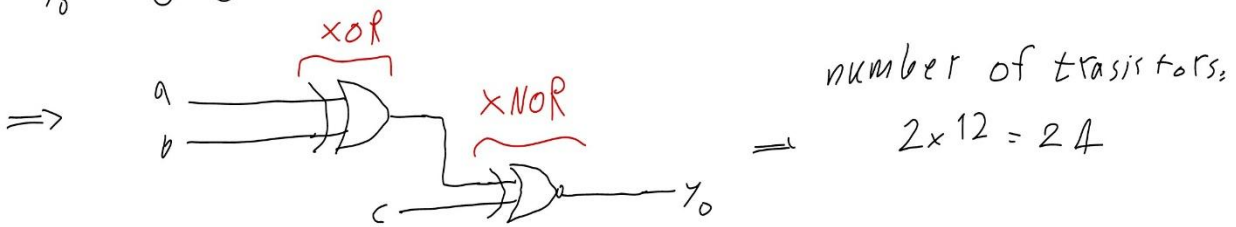
*Implementation details:* This is realized by feeding three 2-input NOR gates into a 3-input NOR gate, which is then passed through a NOT gate.

### 3.3 Gate-Level Schematic

Based on the logic derivation above, the circuit was translated into a gate-level schematic using exclusively standard NOT, NAND, and NOR gates.

Implementing the logics using gate primitives:

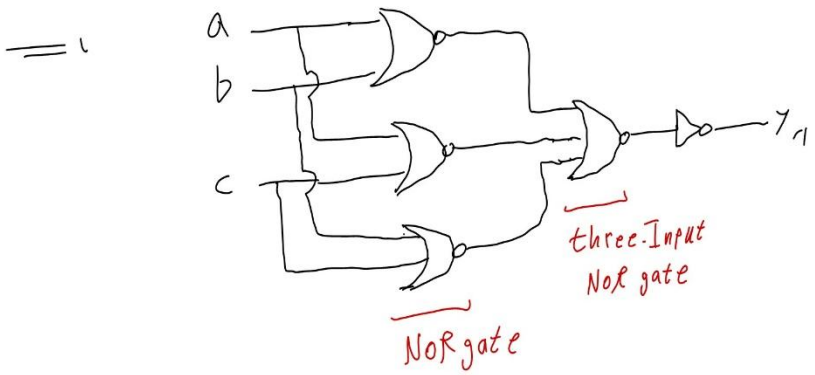
$Y_0 = \overline{a \oplus b \oplus c}$




⇒ number of trasistors:

$2 \times 12 = 24$

$Y_1 = \overline{\overline{(a+b)} + \overline{(a+c)} + \overline{(c+b)}}$




⇒ number of transistors = $3 \times 4 + 6 + 2 = 20$

### 3.4 Delay Modeling and Adjustment

finding structure worst case delay

first we calculat the worst case dilay for gate primitive that we used: pmos #(5,6,7) nmos #(3,4,5)

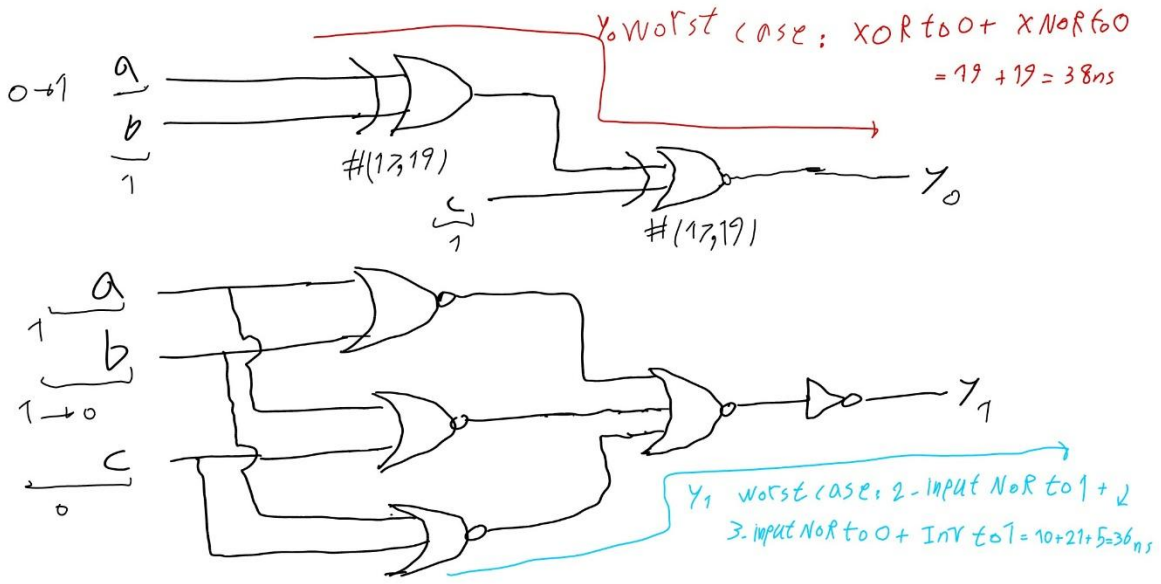
XOR { to1 ⇒ Inv to0 + 2 pmos to1 = 7 + 2×5 = 17ns ; to0 ⇒ Inv to1 + 2 pmos to2 ⇒ 5 + 2×7 = 19 ns }

XNOR { to1 ⇒ Inv to0 + 2 pmos to1 = 7 + 2×5 = 17ns ; to0 ⇒ Inv to1 + 2 pmos to2 = 5 + 2×7 = 19ns }

2-input NOR { to1 ⇒ 2 pmos to1 = 2×5 = 10ns ; to0 ⇒ 2 pmos to2 ⇒ 2×7 = 14ns }

3-input NOR { to1 ⇒ 3 pmos to1 ⇒ 3×5 = 15ns ; to0 ⇒ 3 pmos to2 ⇒ 3×7 = 21 ns }

not { to1 ⇒ nmos to2 ⇒ 5ns ; to0 ⇒ pmos to2 ⇒ 7 ns }




⇒ worst case delay of structure Implemented using gate primitives is: max(38,36) = 38 ns

### 3.5 Verilog Implementation

```
`timescale 1ns/1ns
module ZcGatePrim (input a,b,c , output y1,y0);
    wire nodes[0:4];

    xor #(17,19) G1(nodes[0],a,b);
    xnor #(17,19) G2(y0,nodes[0],c);

    nor #(10,14) G4(nodes[1],a,b);
    nor #(10,14) G5(nodes[2],a,c);
    nor #(10,14) G6(nodes[3],c,b);
    nor #(15,21) G7(nodes[4],nodes[1],nodes[2],nodes[3]);
    not #(5,7) G8(y1,nodes[4]);

endmodule
```

### 3.6 Testbench and Comparative Simulation

**Justifying outputs and worst case value :**

For justifying and worst case delay value we use this test bench:

```
`timescale 1ns/1ns

module ZcGatePrimJustify;

    reg a;
    reg b;
    reg c;

    wire y1_gateprim, y0_gateprim;

    reg [1:0] expected_out;
    real change_time;
    real stable_time;
    real current_delay;
    real worst_delay;

    ZcGatePrim DUT (a, b, c, y1_gateprim, y0_gateprim);

    initial begin
        worst_delay = 0;

        a = 0; b = 0; c = 0;
        expected_out = 2'b11;
```

```
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 1; b = 0; c = 0;
        expected_out = 2'b10;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 1; b = 1; c = 0;
        expected_out = 2'b01;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 0; b = 1; c = 0;
        expected_out = 2'b10;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 0; b = 1; c = 1;
        expected_out = 2'b01;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
```

```
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 0; b = 0; c = 1;
        expected_out = 2'b10;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 1; b = 0; c = 1;
        expected_out = 2'b01;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        a = 1; b = 1; c = 1;
        expected_out = 2'b00;
        change_time = $realtime;
        wait({y1_gateprim, y0_gateprim} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b%b%b | Output: %b%b | Delay: %0t", a, b, c,
y1_gateprim, y0_gateprim, current_delay);
        #60;

        $display("==================================");
        $display("WORST CASE DELAY: %0t", worst_delay);
        $display("==================================");
        $stop;
    end

endmodule
```

justifying simulation result:

```
VSIM 2> run -all
# Input: 000 | Output: 11 | Delay: 36
# Input: 100 | Output: 10 | Delay: 36
# Input: 110 | Output: 01 | Delay: 36
# Input: 010 | Output: 10 | Delay: 36
# Input: 011 | Output: 01 | Delay: 36
# Input: 001 | Output: 10 | Delay: 38
# Input: 101 | Output: 01 | Delay: 36
# Input: 111 | Output: 00 | Delay: 38
# ==================================
# WORST CASE DELAY: 38
# ==================================
# ** Note: $stop    : ZcGatePrimJustify.v(105)
#    Time: 772 ns  Iteration: 0  Instance: /ZcGatePrimJustify
# Break in Module ZcGatePrimJustify at ZcGatePrimJustify.v line 105
```

**Test bench for Comparative simulation:**

```
`timescale 1ns/1ns

module ZcGatePrimTb;

    reg a;
    reg b;
    reg c;

    wire y1_gateprim,y0_gateprim,y1_swichLevel,y0_switchLevel;

    ZcGatePrim DUT (a, b, c, y1_gateprim, y0_gateprim);
    ZC DUT1 (a, b, c, y1_swichLevel, y0_switchLevel);

    initial begin
        a = 0; b = 0; c = 0;
        #60;

        a = 1; b = 0; c = 0;
        #60;

        a = 1; b = 1; c = 0;
        #60;

        a = 0; b = 1; c = 0;
        #60;
```

```
        a = 0; b = 1; c = 1;
        #60;

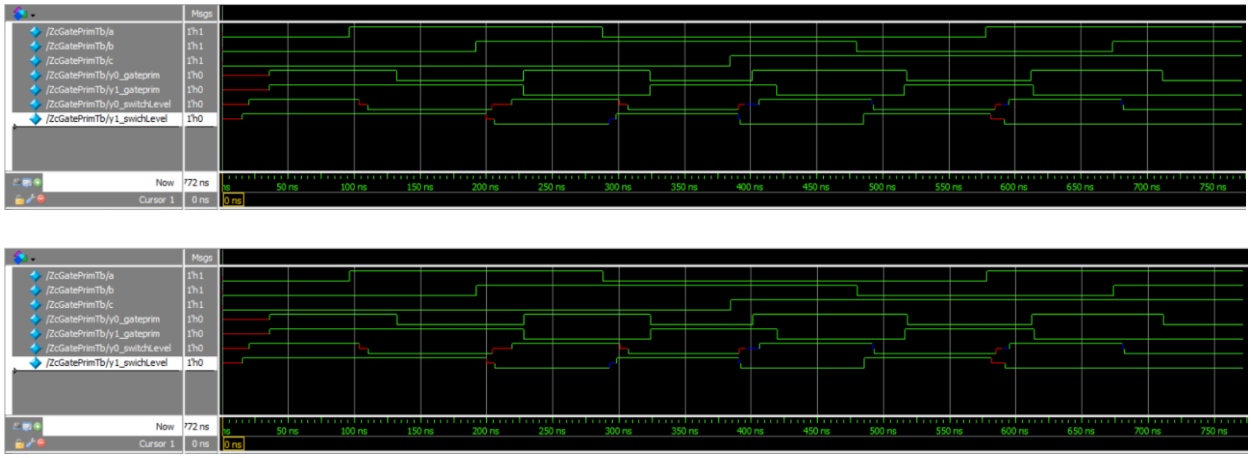
        a = 0; b = 0; c = 1;
        #60;

        a = 1; b = 0; c = 1;
        #60;

        a = 1; b = 1; c = 1;
        #60;
        $stop;
    end

endmodule
```

**Simulation result:**




**Simulation Analysis and Differences:**

Comparing the simulation results of the Zero's Counter implemented with Verilog gate primitives versus switch-level primitives (`pmos`/`nmos`) reveals significant differences in how the simulator models physical hardware behavior.

**1. Presence of Transient States (*X* and *Z*)**

- **Observation:** In the switch-level simulation, the outputs ($Y_1$ and $Y_0$) frequently display temporary $X$(unknown) and $Z$(high-impedance) states during input transitions. In the gate-level simulation, after the initial setup, the outputs transition cleanly between 0and 1without these intermediate states.

- **Why this happens:**
- **Gate-Level:** Verilog standard gate primitives (like `and`, `or`, `not`) operate at a higher level of abstraction. They evaluate pure Boolean logic equations. When inputs change, the gate waits for its specified lumped delay and then instantly updates its output to the new strong 0or 1. It assumes the internal transistor switching is ideal and instantaneous, ignoring intermediate electrical states.
- **Switch-Level:** Switch primitives model the actual Pull-Up Network (PUN) and Pull-Down Network (PDN). Because we assigned asymmetric, realistic delays to the transistors (`pmos` being slower than `nmos`), the simulator accurately models the physical reality that transistors do not turn on and off at exactly the same time. If the PUN and PDN are momentarily ON together due to delay mismatches, it models a short circuit (contention), resulting in an *X*state. If both are momentarily OFF, the output floats, resulting in a *Z*state.

**2. Delay Modeling and Circuit Optimization**

- **Observation:** The overall delay is different between the two models, and gate-level simulation generally results in longer, "worst-case" delays.
- **Why this happens:**
- **Gate-Level (Higher Abstraction = Less Optimization):** When using standard gate primitives (like `and`, `or`, `not`), we are working at a higher level of abstraction using fixed, pre-built blocks. Every block has its own set delay. To build a function, we have to chain several gates together, and all those delays simply add up. Because we cannot change what is inside these pre-built blocks, we cannot optimize them, which leads to longer, worst-case delays.
- **Switch-Level (Lower Abstraction = Better Optimization):** By going down in abstraction to the transistor level, we get complete control over the design. Instead of chaining separate standard gates together, we can design **Complex Gates**. This means we can combine multiple logic operations into one single CMOS transistor structure. Because the signal travels through a single, optimized transistor network rather than multiple separate gates, the circuit is much faster and the delays are shorter and more accurate.

**3. Simulation Performance and Abstraction**

- **Difference:** Gate-level simulation is computationally faster and requires less memory. It evaluates simple truth tables and event queues. Switch-level simulation is much more computationally intensive. The simulator must evaluate bidirectional connections, continuous signal strengths, and individual transistor threshold states, making it a "heavier" simulation reserved for detailed timing and physical verification rather than basic logic checking.

## Part 4: Dataflow Implementation and Comprehensive Comparison

### 4.1 Dataflow Implementation Methodology

In this final phase of the project, the 3-input Zero's Counter (ZC) was modeled using continuous assignment at the dataflow level of abstraction. The goal of this implementation was to encapsulate the boolean logic into Verilog `assign` statements while accurately reflecting the worst-case propagation delays observed in the preceding gate-level model.

Based on the logic derived in earlier sections, the continuous assignments were formulated as follows:

- **Most Significant Bit ($Y_1$):** Implemented using the expression `(~(a | b)) | (~(a | c)) | (~(c | b))`. A lumped inertial delay of `38` time units was applied to match the critical path delay of the gate-level design.
- **Least Significant Bit ($Y_0$):** Implemented using the expression `~(a ^ b ^ c)`. A lumped inertial delay of `36` time units was applied.

The implementation utilizes inertial delays, which are characteristic of continuous assignments in Verilog. Unlike transport delays, inertial delays inherently filter out transient pulses (glitches) that are shorter than the specified delay duration.

**Verilog Implementation:**

```
`timescale  1ns/1ns

module ZC(input a,b,c, output y1,y0);
    assign #(38) y1 = (~(a | b)) | (~(a | c)) | (~(c | b));
    assign #(36) y0= ~(a ^ b ^ c);
endmodule
```

### 4.2 Unified Testbench Design

To facilitate a direct and rigorous comparison across all levels of abstraction, a unified testbench (`ZcGatePrimTb`) was developed. This testbench instantiates the Switch-Level (`ZC`), Gate-Level (`ZcGatePrim`), and Dataflow (`ZcClass`) modules concurrently, stimulating them with identical input vectors. The testbench iterates through all eight possible combinations of the 3-bit input. A time step of 60time units (`#60`) was deliberately chosen between input transitions. This duration exceeds the maximum expected delay (38 time units), ensuring that all models—particularly the switch-level circuit with its dynamic transient states—fully settle to stable values before the subsequent input vector is applied.

Testbench:

```
`timescale 1ns/1ns

module Zc_copmareTb;
```

```
    reg a;
    reg b;
    reg c;

    wire y1_gateprim,y0_gateprim;
    wire y1_swichLevel,y0_switchLevel;
    wire y1_Assigned,y0_Assigned;

    ZcGatePrim DUT (a, b, c, y1_gateprim, y0_gateprim);
    Zc_complexGate DUT1 (a, b, c, y1_swichLevel, y0_switchLevel);
    ZC DUT2 (a, b, c, y1_Assigned, y0_Assigned);

    initial begin
        a = 0; b = 0; c = 0;
        #60;

        a = 1; b = 0; c = 0;
        #60;

        a = 1; b = 1; c = 0;
        #60;

        a = 0; b = 1; c = 0;
        #60;

        a = 0; b = 1; c = 1;
        #60;

        a = 0; b = 0; c = 1;
        #60;

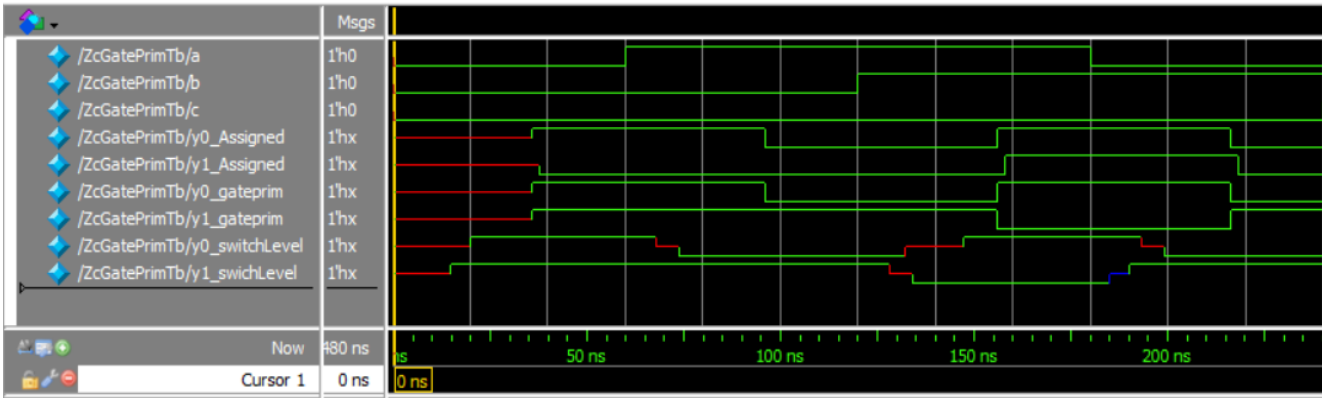
        a = 1; b = 0; c = 1;
        #60;

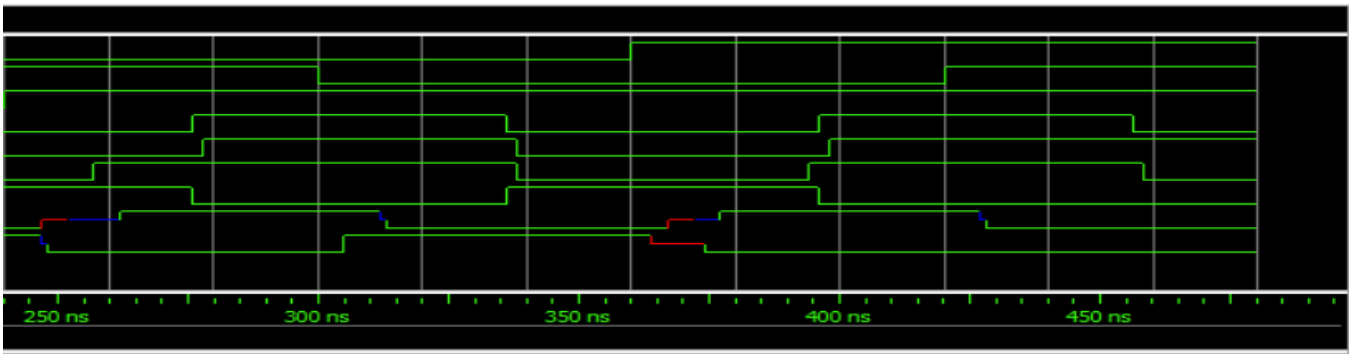
        a = 1; b = 1; c = 1;
        #60;
        $stop;
    end

endmodule
```

### 4.3 Comprehensive Comparative Analysis

**Simulation result :**




**Analysis:**

The parallel simulation of the three distinct abstraction levels reveals critical differences in how digital circuits are modeled and simulated:

- **Transient States ($X$ and $Z$):** The Switch-Level model exhibits unknown ($X$) and high-impedance ($Z$) states during transitions. As analyzed in Part 2, these are direct physical manifestations of the asymmetric pull-up and pull-down network delays (nMOS versus pMOS). In contrast, both the Gate-Level and Dataflow models abstract away transistor-level physics, resulting in idealized transitions directly between 0and 1without contention or floating states.
- **Hazard and Glitch Propagation:** While the Gate-Level model utilizes lumped delays at the gate level, varying path lengths can still result in transient logic hazards (glitches) before the final steady-state output is reached. The Dataflow model, however, utilizes single continuous assignments with inertial delays. Consequently, any input fluctuations shorter than the lumped delay (e.g., $<38$time units for $Y_1$) are absorbed, yielding significantly cleaner waveform transitions compared to the structural models.
- **Abstraction Trade-offs:**
- **Switch-Level:** Provides the highest physical fidelity and accurately models analog-like timing characteristics, but incurs the highest computational cost during simulation.

- **Gate-Level:** Offers an intermediate structural representation, balancing the representation of internal propagation paths with faster simulation speeds.
- **Dataflow Level:** Represents the highest level of abstraction in this project. It is the most computationally efficient for simulation and provides the cleanest output by filtering physical anomalies, though it completely masks the internal structural timing intricacies of the physical circuit.

# 5. 7-Input Zero's Counter

## 5.1. Objective

The objective of this section is to design a 7-input Zero's Counter (data_in[0:6]) by cascading multiple instances of the previously designed 3-input Zero's Counter (Zc_complexGate). This demonstrates hierarchical structural modeling and an optimized approach to modular logic reuse.

## 5.2. Architecture and Logic Design

A 7-input ZC requires a 3-bit output (msb_out, mid_out, lsb_out) to represent a maximum count of 7. The design uses a tree-like adder architecture divided into distinct grouping and addition stages.

The architecture handles the intermediate signals using a highly optimized inversion strategy:

1. **First Stage (Grouping):**

- stage1_gate0 processes the first three inputs (data_in[0:2]), outputting stage1_msb[0] and stage1_lsb[0].
- stage1_gate1 processes the next three inputs (data_in[3:5]), outputting stage1_msb[1] and stage1_lsb[1].

2. **Second Stage (LSB Addition):**

- Because the ZC modules count 0s, an active count is represented by a logic 1. To properly add these counts in the next stage, the LSBs from the first stage must be inverted back to 0s.
- inv_stage1_a and inv_stage1_b perform this inversion with specific rise/fall delays of #(7,5).
- stage2_gate0 adds these inverted LSBs along with the raw 7th input (data_in[6]). Its LSB output becomes the final lsb_out of the system. Its MSB output acts as a carry (stage1_msb[2]) for the MSB stage.

3. **Third Stage (MSB Addition):**

- Instead of inverting the three MSB signals (stage1_msb[0], stage1_msb[1], and the carry stage1_msb[2]) before feeding them into stage2_gate1, the design feeds them directly.
- Because stage2_gate1 is a *zeros* counter, feeding it raw 1s means it effectively counts the *absence* of MSB carries.

- To correct this logic, the outputs of stage2_gate1 (stage1_msb[3] and stage1_lsb[2]) are inverted at the very end using inv_msb and inv_mid with #(7,5) delays.

### 5.3. Structural Verilog Implementation

The implementation is purely structural, instantiating Zc_complexGate modules and not primitives .

```
`timescale 1ns/1ns

module seven_input_zc(
    input [0:6] data_in,
    output msb_out, mid_out, lsb_out
);
    wire [0:3] stage1_msb;
    wire [0:2] stage1_lsb;
    wire stage2_inv_a, stage2_inv_b;

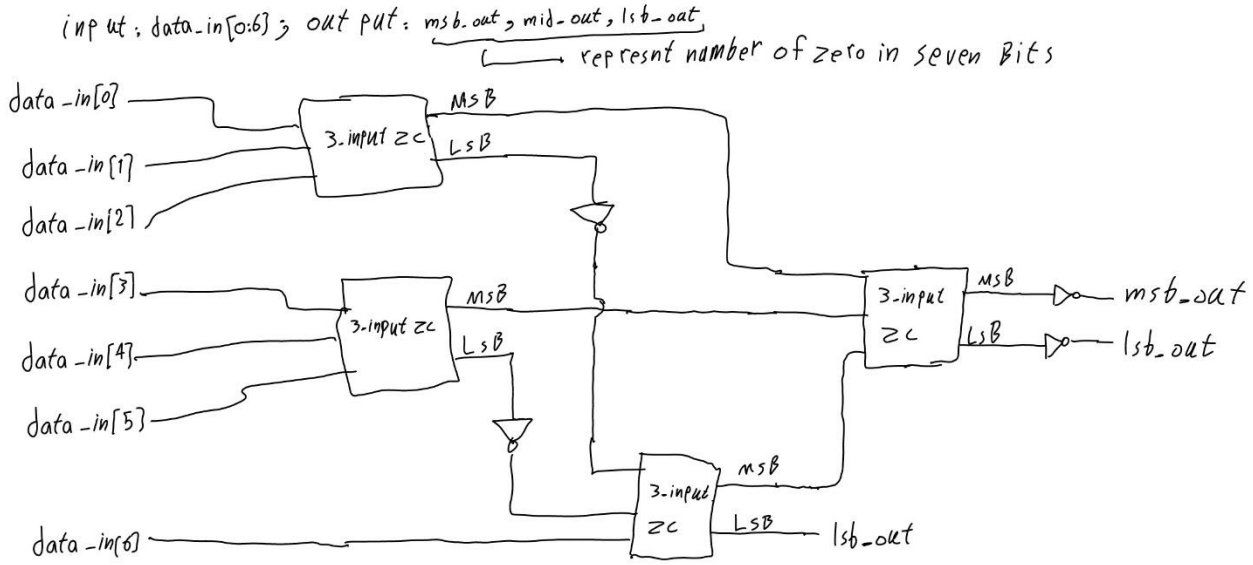
    Zc_complexGate stage1_gate0(data_in[0], data_in[1], data_in[2],
                                stage1_msb[0], stage1_lsb[0]);
    Zc_complexGate stage1_gate1(data_in[3], data_in[4], data_in[5],
                                stage1_msb[1], stage1_lsb[1]);

    not #(5,7) inv_stage1_a(stage2_inv_a, stage1_lsb[0]);
    not #(5,7) inv_stage1_b(stage2_inv_b, stage1_lsb[1]);

    Zc_complexGate stage2_gate0(stage2_inv_b,stage2_inv_a,data_in[6],
                                stage1_msb[2], lsb_out);
    Zc_complexGate stage2_gate1(stage1_msb[1], stage1_msb[2], stage1_msb[0],
                                stage1_msb[3], stage1_lsb[2]);

    not #(5,7) inv_mid(mid_out, stage1_lsb[2]);
    not #(5,7) inv_msb(msb_out, stage1_msb[3]);
endmodule
```

### 5.4 Delay Modeling and Adjustment




calculating delay:

for 3-input zc we used the complex gate structure that we built in part 2.

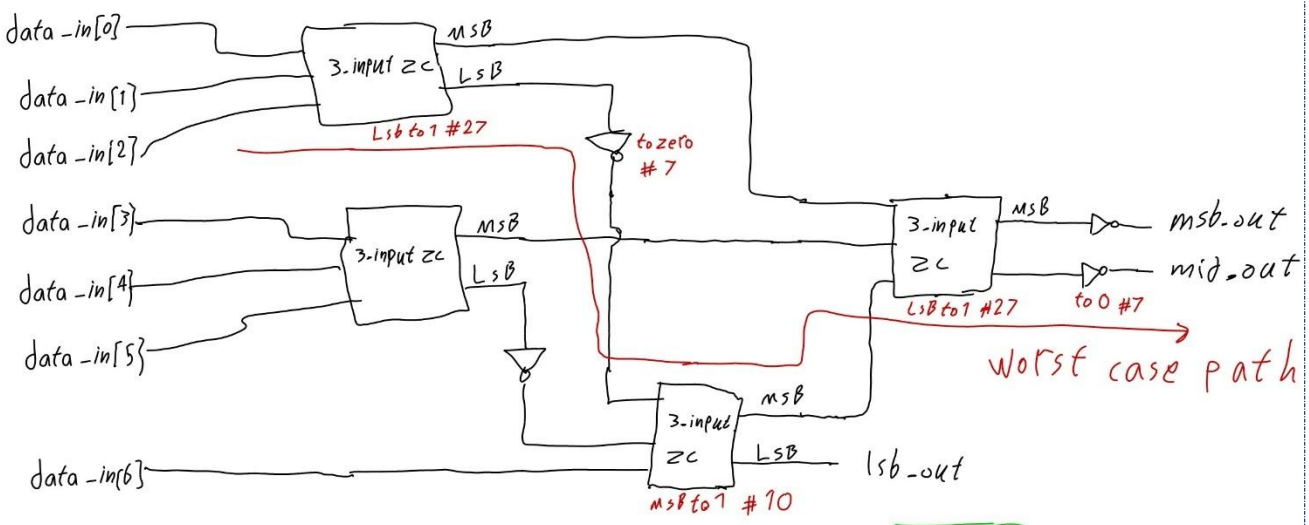
⟹ so the worst case delay of 3-input zc is:

for LSB: { to 1: 27ns ; to 0: 21ns }

for MSB: { to 1: 10ns ; to 0: 14ns }

⟶ for Inverter as we calculated in part 3 we have: # (5,7)

for transision 1001000 → 1101000 we have:




⟹ the worstcase delay is: 27×2 + 7×2 + 10 = 78 ns

### 5.5. Simulation and Analysis

***Testbench implementation :***

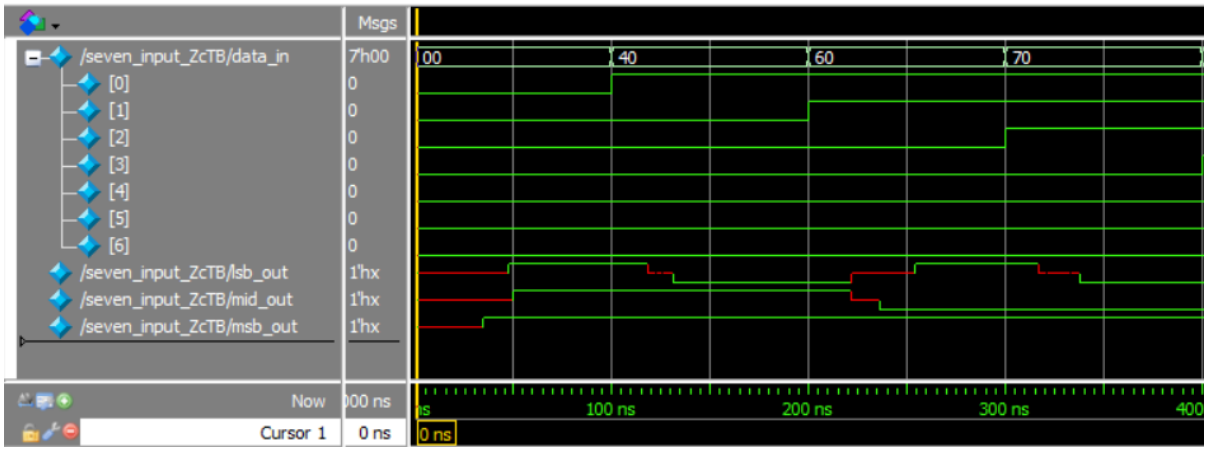
```
`timescale 1ns/1ns

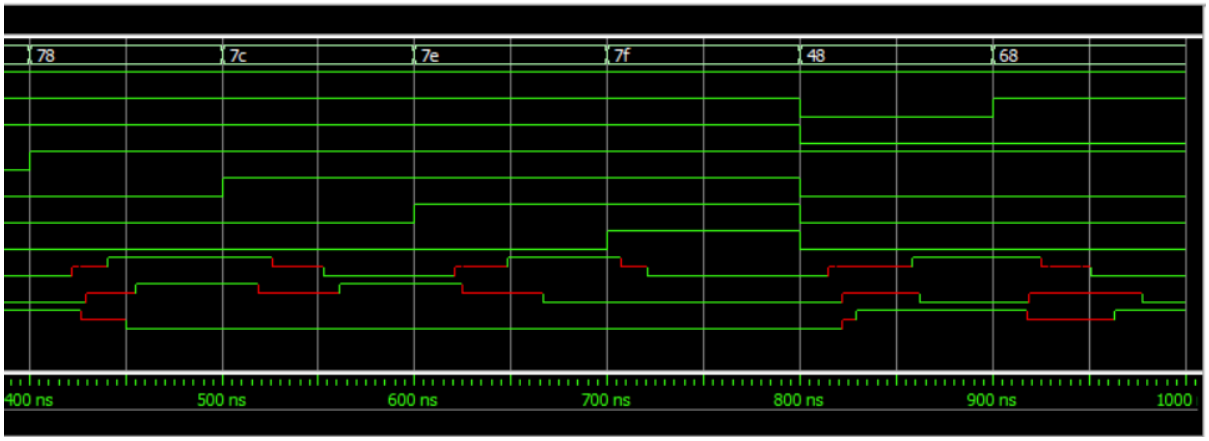
module seven_input_zcTB;
    reg [0:6] data_in;
    wire msb_out, mid_out, lsb_out;

    seven_input_zc DUT (
        .data_in(data_in),
        .msb_out(msb_out),
        .mid_out(mid_out),
        .lsb_out(lsb_out)
    );

    initial begin
        data_in = 7'b0000000;
        #100;
        data_in = 7'b1000000;
        #100;
        data_in = 7'b1100000;
        #100;
        data_in = 7'b1110000;
        #100;
        data_in = 7'b1111000;
        #100;
        data_in = 7'b1111100;
        #100;
        data_in = 7'b1111110;
        #100;
        data_in = 7'b1111111;
        #100;
        data_in = 7'b1001000;
        #100;
        data_in = 7'b1101000;
        #100;
        $stop;
    end
endmodule
```

**simulation result :**




**Output and worst case delay justifying:**

**we use this test bench for justifying worst case delay and outputs:**

```
`timescale 1ns/1ns

module seven_input_zcJustify;
    reg [0:6] data_in;
    wire msb_out, mid_out, lsb_out;

    reg [2:0] expected_out;
    real change_time;
    real stable_time;
    real current_delay;
    real worst_delay;

    seven_input_zc DUT (
        .data_in(data_in),
        .msb_out(msb_out),
        .mid_out(mid_out),
        .lsb_out(lsb_out)
    );
```

```
    initial begin
        worst_delay = 0;

        data_in = 7'b0000000;
        expected_out = 3'b111;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1000000;
        expected_out = 3'b110;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1100000;
        expected_out = 3'b101;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1110000;
        expected_out = 3'b100;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1111000;
```

```
        expected_out = 3'b011;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1111100;
        expected_out = 3'b010;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1111110;
        expected_out = 3'b001;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1111111;
        expected_out = 3'b000;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1001000;
        expected_out = 3'b101;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
```

```
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        data_in = 7'b1101000;
        expected_out = 3'b100;
        change_time = $realtime;
        wait({msb_out, mid_out, lsb_out} == expected_out);
        stable_time = $realtime;
        current_delay = stable_time - change_time;
        if (current_delay > worst_delay) worst_delay = current_delay;
        $display("Input: %b | Output: %b%b%b | Delay: %0t", data_in, msb_out,
mid_out, lsb_out, current_delay);
        #100;

        $display("================================");
        $display("WORST CASE DELAY: %0t", worst_delay);
        $display("================================");
        $stop;
    end
endmodule
```

**justifying result:**

```
# Input: 0000000 | Output: 111 | Delay: 50
# Input: 1000000 | Output: 110 | Delay: 31
# Input: 1100000 | Output: 101 | Delay: 54
# Input: 1110000 | Output: 100 | Delay: 38
# Input: 1111000 | Output: 011 | Delay: 55
# Input: 1111100 | Output: 010 | Delay: 61
# Input: 1111110 | Output: 001 | Delay: 67
# Input: 1111111 | Output: 000 | Delay: 21
# Input: 1001000 | Output: 101 | Delay: 62
# Input: 1101000 | Output: 100 | Delay: 78
# ================================
# WORST CASE DELAY: 78
# ================================
# ** Note: $stop    : seven_input_zcTb.v(126)
#    Time: 1517 ns  Iteration: 0  Instance: /seven_input_zcTB
# Break in Module seven_input_zcTB at seven_input_zcTb.v line 126
```

## Part 6: 7-Input ZC (Dataflow) and Timing Comparison

### 6.1. Objective and Implementation

The objective of this section is to implement the 7-input Zero's Counter using the dataflow-level abstraction (`ZcClass` modules) and compare its timing and transient behavior against the complex gate-level implementation designed in Part 5.

The architecture for the dataflow version (`seven_input_Zc_assignedPrim`) directly mirrors the optimized hierarchical tree structure used in Part 5. It instantiates the `ZcClass` modules (which utilize continuous `assign` statements with lumped delays of `#(38)` and `#(36)`) for the intermediate stages and relies on standard inverters to finalize the output logic.

### 6.2. Comparative Testbench

To accurately observe the differences between the two abstraction levels, a unified testbench (`Zc_7input_compare_tb`) was utilized. This testbench instantiates both the `seven_input_Zc` (complex gate) and the `seven_input_Zc_assignedPrim` modules side-by-side. By applying identical input vectors with sufficient time spacing (e.g., `#100`), we can overlay their outputs and directly compare their timing profiles on the waveform viewer.

Here is the test bench:

```
`timescale 1ns/1ns

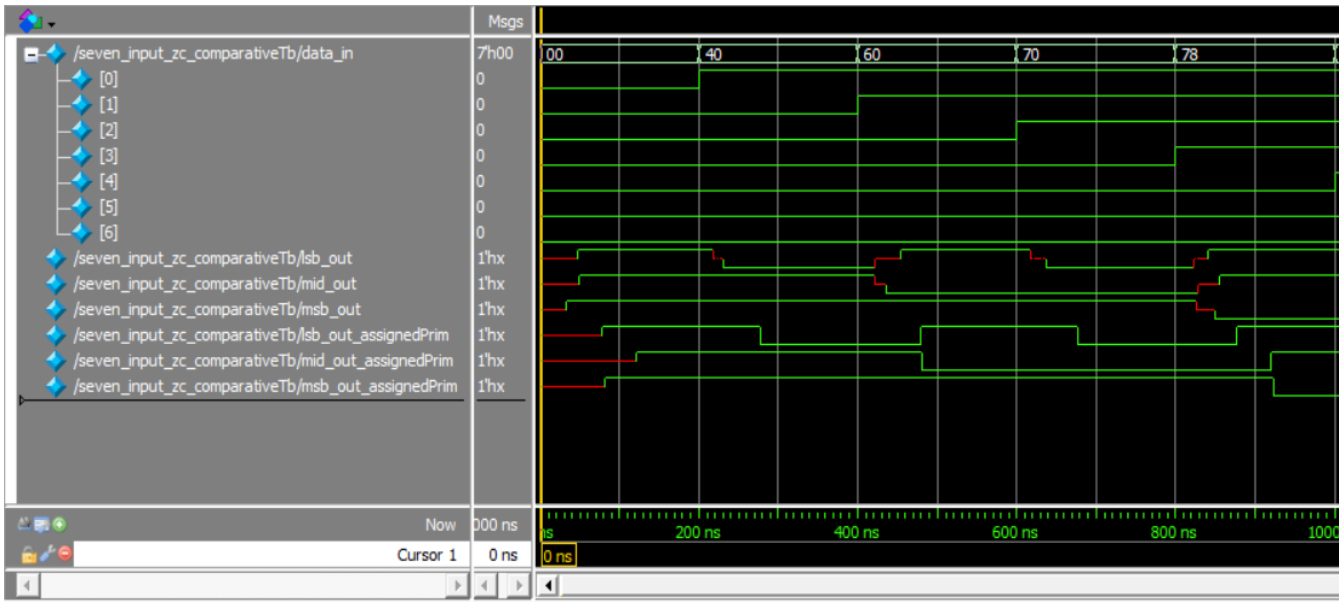
module seven_input_zc_comparativeTb;
    reg [0:6] data_in;
    wire msb_out, mid_out, lsb_out;
    wire msb_out_assignedPrim,mid_out_assignedPrim,lsb_out_assignedPrim;

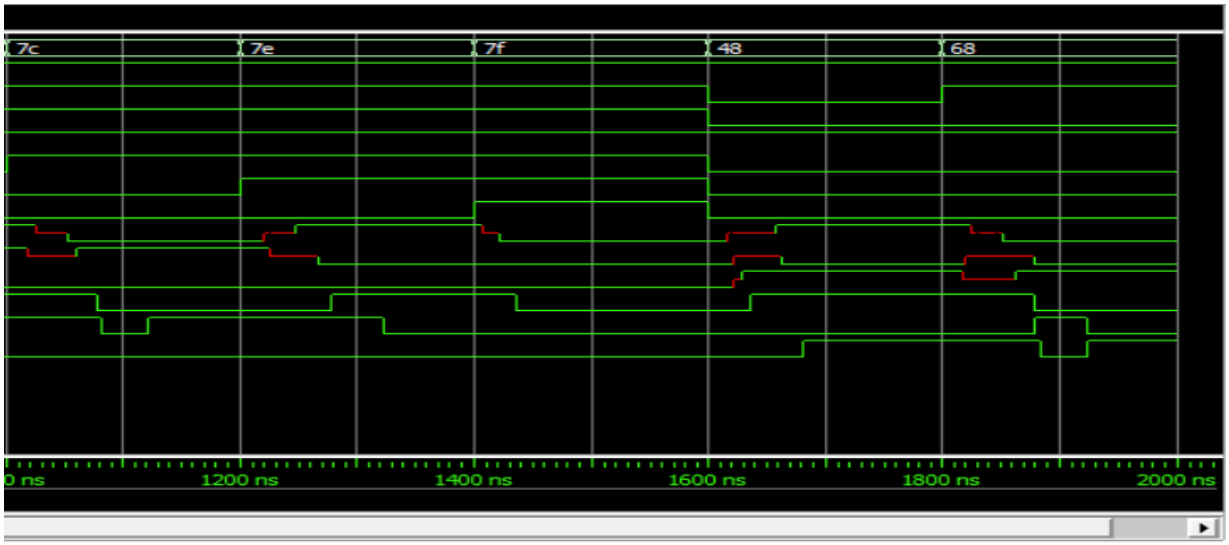
    seven_input_zc DUT (
        .data_in(data_in),
        .msb_out(msb_out),
        .mid_out(mid_out),
        .lsb_out(lsb_out)
    );
    seven_input_zc DUT (
        .data_in(data_in),
        .msb_out_assignedPrim(msb_out),
        .mid_out_assignedPrim(mid_out),
        .lsb_out(lsb_out_assignedPrim)
    );

    initial begin
        data_in = 7'b0000000;
        #100;
        data_in = 7'b1000000;
        #100;
        data_in = 7'b1100000;
```

```
        #100;
        data_in = 7'b1110000;
        #100;
        data_in = 7'b1111000;
        #100;
        data_in = 7'b1111100;
        #100;
        data_in = 7'b1111110;
        #100;
        data_in = 7'b1111111;
        #100;
        data_in = 7'b1001000;
        #100;
        data_in = 7'b1101000;
        #100;
        $stop;
    end
endmodule
```

### 6.3. Simulation Results and Comparative Analysis






Based on the comparative simulation waveforms and the design process, several key observations define the differences between the dataflow (`assign`) and complex gate implementations:

1. **Clean Outputs and Glitch Absorption:** In the dataflow model using `assign` statements, we do not observe any transient *X*(unknown) or *Z*(high-impedance) states. The inertial delays used in the continuous assignments effectively "absorb" or filter out internal glitches and hazards. As a result, the dataflow version produces perfectly clean, instantaneous transitions between standard binary states.
2. **Pessimistic Delay Accumulation:** A major observation from the waveforms is that the overall delays and the worst-case delay in the `assign` implementation are *greater* than those in the complex gate build. Because the dataflow model rigidly applies lumped, maximum-possible delays at every single stage, this pessimism compounds through the hierarchy. The complex gate model, with its distributed internal node delays, often resolves faster in reality than the static lumped equations suggest.
3. **Simplicity and High Abstraction:** Despite the overestimated delays, the `assign`-based model is significantly simpler to build, code, and analyze. By operating at a higher level of abstraction, it removes the need to worry about transistor contention, asymmetric rise/fall times, or internal node capacitance. It provides a straightforward, easy-to-read mathematical representation of the circuit, making functional analysis much simpler than tracing paths in the complex gate schematic.

**Conclusion for Part 6:**

The dataflow model is highly abstract and much simpler to build, offering clean, glitch-free outputs that are very easy to analyze logically. However, this simplicity comes at the cost of timing accuracy; it statically forces pessimistic worst-case delays, resulting in a slower overall measured delay compared to the physically detailed complex gate implementation.

## Part 7: 15-Input Zero's Counter Implementation

### 7.1 Objective

The objective of this section is to design and implement a 15-input Zero's Counter (_fifteen_input_zc). To manage the complexity of counting zeros across 15 parallel lines, a hierarchical design approach is utilized. Instead of building the logic from scratch, the design reuses previously constructed and verified modules: the seven_input_zc (7-input Zero's Counter) and the Zc_complexGate (3-input Zero's Counter).

### 7.2 Design Architecture and Logic

The core idea behind the 15-input Zero's Counter is identical to the strategy used when building the 7-input Zero's Counter: divide the input bits into manageable groups, count the zeros in each group independently, and then sum the intermediate counts.

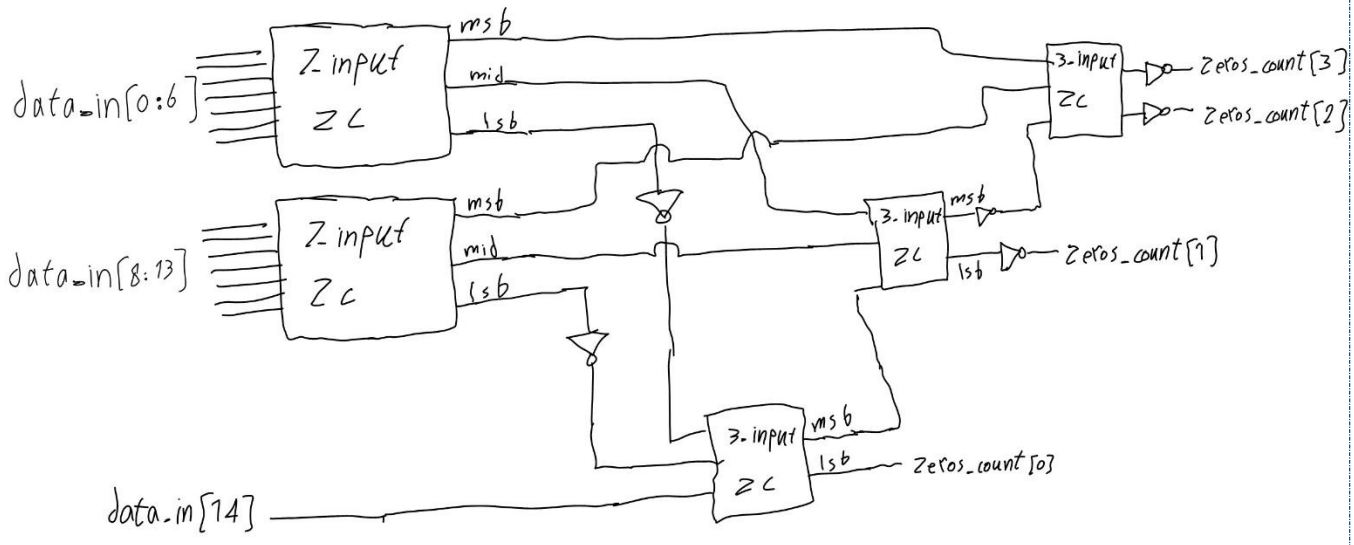
For a 15-bit input (15 bits total), the architecture is divided as follows:

1. **Division:** The first 14bits are split evenly into two groups of 7bits. The remaining 1bit (data_in[14]) is set aside to be added later.
2. **Intermediate Counting:** Two instances of the seven_input_zc module are used.
    - Instance A processes data_in[6:0] and outputs a 3-bit count (count_a).
    - Instance B processes data_in[13:7] and outputs a 3-bit count (count_b).
3. **Summation (Building an Adder):** To get the final 4-bit count, we must add count_a, count_b, and the remaining 15th bit (data_in[14]). Since the Zc_complexGate functions identically to a 1-bit Full Adder when its outputs are inverted, we cascade three of these modules to form a 3-bit ripple-carry adder:
    - **Bit 0 Stage:** The LSBs of the two counts (count_a[0] and count_b[0]) are inverted and fed into the first Zc_complexGate along with the 15th input bit (data_in[14]). This stage directly yields the final LSB of the total count (zeros_count[0]) and the first carry (c1).
    - **Bit 1 Stage:** The middle bits (count_a[1] and count_b[1]) and the carry c1 are fed into the second Zc_complexGate. The outputs are inverted using not #(5,7) gates to produce zeros_count[1] and the next carry c2.
    - **Bit 2 Stage:** The MSBs (count_a[2] and count_b[2]) and the carry c2 are fed into the third Zc_complexGate. Inverting the outputs produces zeros_count[2] and the final carry, which acts as the MSB of the total count (zeros_count[3]).

### 7.3 Schematic and Delay Modeling and Adjustment

15_input zero counter

input: data_in [0:14] ⟹ output zeros_count [0:3]

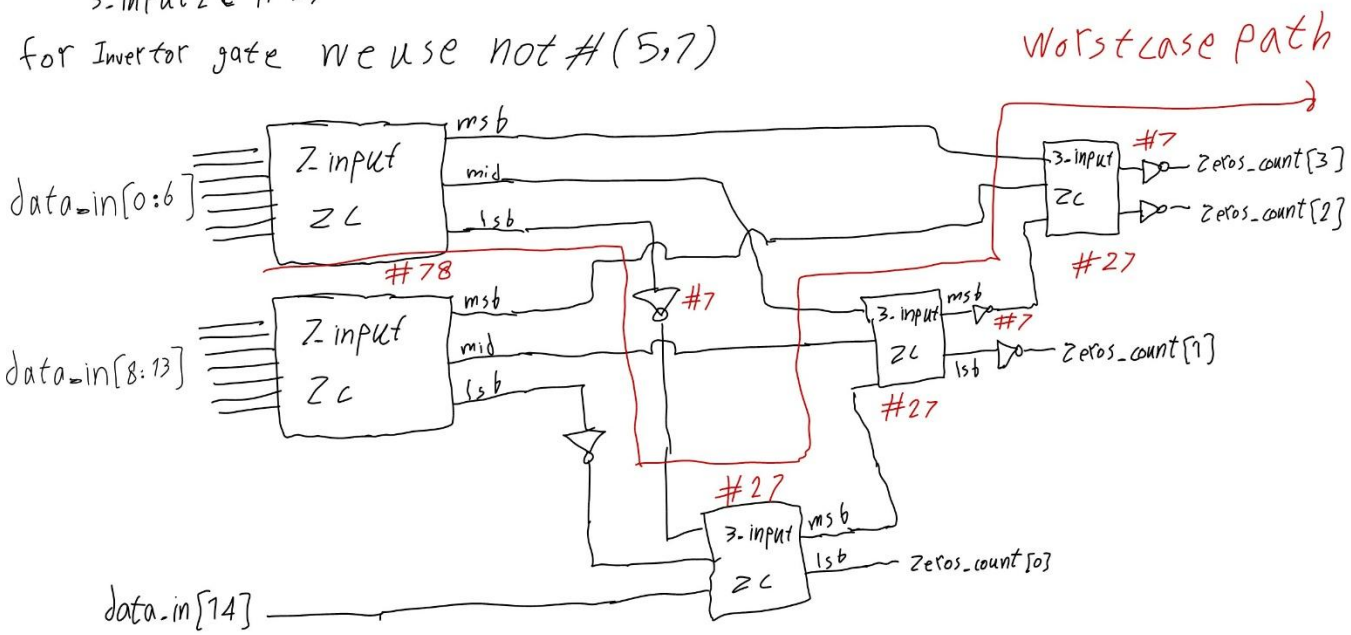



calculating worstcase delay: for 3_input and 7_input we use complex gate structure of them so as we calculated in previous parts:

7_input zc #78

3_input zc #27

for Invertor gate we use not #(5,7)




⟹ so the worst case delay is: 78 + 3x7 + 3x27 = 180 ns

### 7.4 Structural Verilog Implementation

The logic described above is implemented using purely structural Verilog, strictly utilizing the designated gate delays (not #(5,7)) and instantiated modules.

```
`timescale 1ns/1ns


module _fifteen_input_zc(
    input  wire [14:0] data_in,
    output wire [3:0]  zeros_count
);

    wire [2:0] count_a;
    wire [2:0] count_b;

    wire c1, c2, c3;

    wire not_count_a0, not_count_b0;

    wire y1_stage1, y0_stage1;
    wire y1_stage2, y0_stage2;

    seven_input_zc zc7_inst_A (
        .data_in(data_in[6:0]),
        .msb_out(count_a[2]),
        .mid_out(count_a[1]),
        .lsb_out(count_a[0])
    );

    seven_input_zc zc7_inst_B (
        .data_in(data_in[13:7]),
        .msb_out(count_b[2]),
        .mid_out(count_b[1]),
        .lsb_out(count_b[0])
    );


    not #(5,7) inv_a0 (not_count_a0, count_a[0]);
    not #(5,7) inv_b0 (not_count_b0, count_b[0]);

    Zc_complexGate adder_bit0 (
        .a(not_count_a0),
        .b(not_count_b0),
        .c(data_in[14]),
        .y1(c1),
        .y0(zeros_count[0])
    );

    Zc_complexGate adder_bit1 (
```

```
        .a(count_a[1]),
        .b(count_b[1]),
        .c(c1),
        .y1(y1_stage1),
        .y0(y0_stage1)
    );

    not #(5,7) inv_sum1 (zeros_count[1], y0_stage1);
    not #(5,7) inv_c2   (c2, y1_stage1);

    Zc_complexGate adder_bit2 (
        .a(count_a[2]),
        .b(count_b[2]),
        .c(c2),
        .y1(y1_stage2),
        .y0(y0_stage2)
    );

    not #(5,7) inv_sum2 (zeros_count[2], y0_stage2);
    not #(5,7) inv_c3   (zeros_count[3], y1_stage2);

endmodule
```

### 7.5 Simulation and Verification

To verify the correct functionality and analyze the timing of the 15-input Zero's Counter, a testbench was created to apply various test vectors.

**Test bench:**

```
`timescale 1ns/1ns

module fifteen_input_zc_tb;
    reg [14:0] data_in;
    wire [3:0] zeros_count;

    _fifteen_input_zc DUT (
        .data_in(data_in),
        .zeros_count(zeros_count)
    );

    initial begin
        $display("Time\tInput\t\tZeros\tExpected");
        $display("----\t-----\t\t-----\t--------");

        data_in = 15'b000000000000000;
        #200;
        $display("%0t\t%b\t%b\t1111", $time, data_in, zeros_count);
```

```
        data_in = 15'b100000000000000;
        #200;
        $display("%0t\t%b\t%b\t1110", $time, data_in, zeros_count);

        data_in = 15'b110000000000000;
        #200;
        $display("%0t\t%b\t%b\t1101", $time, data_in, zeros_count);

        data_in = 15'b111000000000000;
        #200;
        $display("%0t\t%b\t%b\t1100", $time, data_in, zeros_count);

        data_in = 15'b111100000000000;
        #200;
        $display("%0t\t%b\t%b\t1011", $time, data_in, zeros_count);

        data_in = 15'b111110000000000;
        #200;
        $display("%0t\t%b\t%b\t1010", $time, data_in, zeros_count);

        data_in = 15'b111111000000000;
        #200;
        $display("%0t\t%b\t%b\t1001", $time, data_in, zeros_count);

        data_in = 15'b111111100000000;
        #200;
        $display("%0t\t%b\t%b\t1000", $time, data_in, zeros_count);

        data_in = 15'b111111110000000;
        #200;
        $display("%0t\t%b\t%b\t0111", $time, data_in, zeros_count);

        data_in = 15'b111111111000000;
        #200;
        $display("%0t\t%b\t%b\t0110", $time, data_in, zeros_count);

        data_in = 15'b111111111100000;
        #200;
        $display("%0t\t%b\t%b\t0101", $time, data_in, zeros_count);

        data_in = 15'b111111111110000;
        #200;
        $display("%0t\t%b\t%b\t0100", $time, data_in, zeros_count);

        data_in = 15'b111111111111000;
        #200;
        $display("%0t\t%b\t%b\t0011", $time, data_in, zeros_count);
```

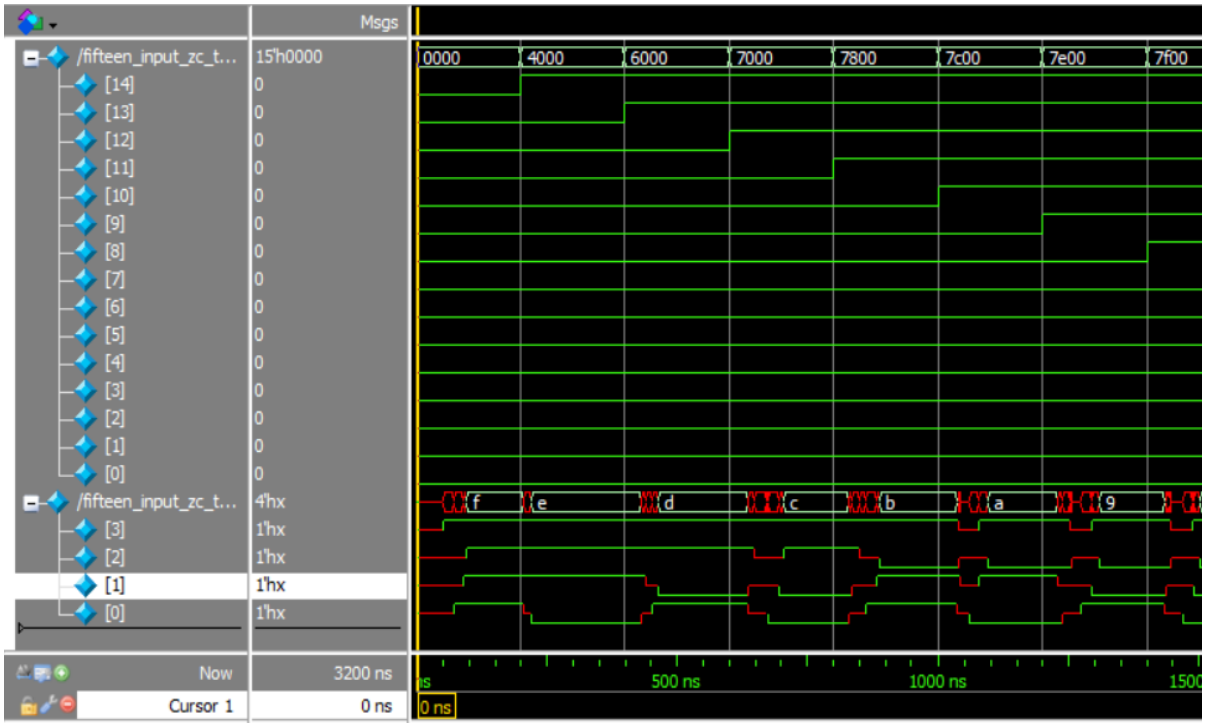
```
        data_in = 15'b111111111111100;
        #200;
        $display("%0t\t%b\t%b\t0010", $time, data_in, zeros_count);

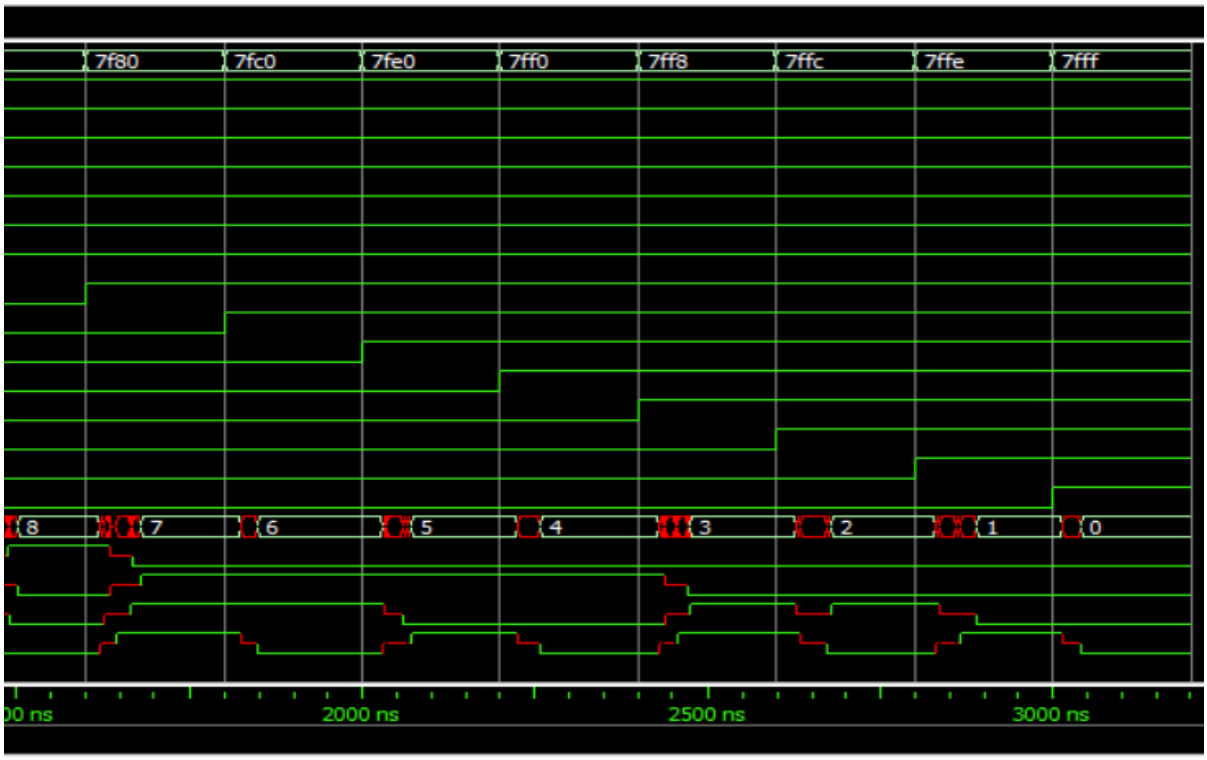
        data_in = 15'b111111111111110;
        #200;
        $display("%0t\t%b\t%b\t0001", $time, data_in, zeros_count);

        data_in = 15'b111111111111111;
        #200;
        $display("%0t\t%b\t%b\t0000", $time, data_in, zeros_count);

        $stop;
    end
endmodule
```

**Simulation result:**






**Justify outputs:**

```
VSIM 3> run -all
# Time   Input            Zeros   Expected
# ----   -----            -----   --------
# 200    0000000000000000 1111 1111
# 400    1000000000000000 1110 1110
# 600    1100000000000000 1101 1101
# 800    1110000000000000 1100 1100
# 1000   1111000000000000 1011 1011
# 1200   1111100000000000 1010 1010
# 1400   1111110000000000 1001 1001
# 1600   1111111000000000 1000 1000
# 1800   1111111100000000 0111 0111
# 2000   1111111110000000 0110 0110
# 2200   1111111111000000 0101 0101
# 2400   1111111111100000 0100 0100
# 2600   1111111111110000 0011 0011
# 2800   1111111111111100 0010 0010
# 3000   1111111111111110 0001 0001
# 3200   1111111111111111 0000 0000
# ** Note: $stop    : fifteen_input_zc_tb.v(80)
#    Time: 3200 ns  Iteration: 0  Instance: /fifteen_input_zc_tb
# Break in Module fifteen_input_zc_tb at fifteen_input_zc_tb.v line 80
```

### 7.6 Conclusion

The 15-input Zero's Counter was successfully implemented by leveraging a hierarchical structural design. By reusing the 7-input counter and adapting the 3-input complex gate to function as a full adder, the complex task of tallying 15bits was broken down into simple, combinational addition stages. The simulation results confirm that the circuit functions perfectly as a truth-table-accurate Zero's Counter, while the timing analysis highlights the cumulative propagation delays inherent in deep structural hierarchies.